

Create Observer

1-Interface

```
1 package com.pisethjava.observer.classic;
2
3 @FunctionalInterface
4 public interface UserObserver {
5     void onUserRegistered(UserRegisteredEvent event);
6
7 }
8
```

2-Implementation

```
UserService.java ClassicObserverDemo.java AuditObserver.java UserObserver.java AnalyticsObserver.java X
observer-pattern-plain-java > src > com.pisethjava.observer.classic > AnalyticsObserver >
1 package com.pisethjava.observer.classic;
2
3 public class AnalyticsObserver implements UserObserver {
4
5     @Override
6     public void onUserRegistered(UserRegisteredEvent event) {
7         System.out.println("METRIC -> Increment user.registration.count");
8     }
9
10 }
11 }
```

```
UserService.java ClassicObserverDemo.java AuditObserver.java X UserObserver.java AnalyticsObserver.java
observer-pattern-plain-java > src > com.pisethjava.observer.classic > AuditObserver >
1 package com.pisethjava.observer.classic;
2
3 public class AuditObserver implements UserObserver {
4
5     @Override
6     public void onUserRegistered(UserRegisteredEvent event) {
7         System.out.println("AUDIT -> eventId= " + event.user().username());
8     }
9
10 }
11 }
```

```

UserService.java ClassicObserverDemo.java AuditObserver.java UserObserver.java AnalyticsObserver.java SmsObserver.java X
observer-pattern-plain-java > src > com.pisethjava.observer.classic > SmsObserver >
1 package com.pisethjava.observer.classic;
2
3 public class SmsObserver implements UserObserver {
4
5     @Override
6     public void onUserRegistered(UserRegisteredEvent event) {
7         System.out.println("SMS -> Send message to " + event.user().phone());
8     }
9 }
10
11 }

```

```

WelcomeEmailObserver.java X
observer-pattern-plain-java > src > com.pisethjava.observer.classic > WelcomeEmailObserver >
1 package com.pisethjava.observer.classic;
2
3 public class WelcomeEmailObserver implements UserObserver {
4
5     @Override
6     public void onUserRegistered(UserRegisteredEvent event) {
7         System.out.println("EMAIL -> Welcome " + event.user().email());
8     }
9 }
10
11 }

```

3-Crete model

```

UserEventPublisher.java UserService.java User.java X
observer-pattern-plain-java > src > com.pisethjava.observer.domain > User >
1 package com.pisethjava.observer.domain;
2
3 public record User(String username, String email, String phone) {
4     public User {
5         if(username == null || username.isBlank()) {
6             throw new IllegalArgumentException("Username is required");
7         }
8
9         if(email == null || email.isBlank()) {
10            throw new IllegalArgumentException("Email is required");
11        }
12    }
13 }
14

```

4-Service

```
UserService.java X
observer-pattern-plain-java > src > com.pisethjava.observer.classic > UserService >
1 package com.pisethjava.observer.classic;
2
3 import com.pisethjava.observer.domain.User;
4
5 public class UserService {
6
7     private final UserEventPublisher publisher;
8
9     public UserService(UserEventPublisher publisher) {
10         this.publisher = publisher;
11     }
12
13     public User register(User user) {
14         System.out.println("SERVICE -> Save user " + user.username());
15
16         // where we publish the user to ousite
17         publisher.publish(new UserRegisteredEvent(user));
18         return user;
19     }
20 }
21 }
```

- UserService សម្រាប់ធ្វើការទាំងឡាយណាដែលទាក់ទងនឹងវា

```
public User register(User user) {
    System.out.println("SERVICE -> Save user " + user.username());

    // where we publish the user to ousite
    publisher.publish(new UserRegisteredEvent(user));
    return user;
}
```

វាធ្វើការ Register User ដែលយើងធ្វើការ បោះ User ចូលហើយវាធ្វើការតែការចុះឈ្មោះនឹងនោះទេ
ពេលវាធ្វើហើយ វា Publish User នឹងអោយទៅខាងក្រៅទៅចប់ហើយ

```
// where we publish the user to ousite
publisher.publish(new UserRegisteredEvent(user));
```

ដែលសម្រាប់ការធ្វើ Publisher នឹងគឺយើងប្រើប្រាស់ Server UserEventPublisher

```
private final UserEventPublisher publisher;
```

```

1 package com.pisethjava.observer.classic;
2
3 import java.util.List;
4
5
6 public class UserEventPublisher {
7     // register observer
8     private final List<UserObserver> observers = new CopyOnWriteArrayList<>();
9
10    // CopyOnWriteArrayList<>() for thread safe
11
12    public void subscribe(UserObserver observer) {
13        if (observer == null) {
14            throw new IllegalArgumentException("Observer is required");
15        }
16        observers.add(observer);
17    }
18
19    public void unsubscribe(UserObserver observer) {
20        observers.remove(observer);
21    }
22
23    public void publish(UserRegisteredEvent event) {
24        for (UserObserver observer : observers) {
25            try {
26                observer.onUserRegistered(event);
27            } catch (RuntimeException exception) {
28                System.out.printf("Observer %s failed: %s%n",
29                    observer.getClass().getSimpleName(), exception.getMessage());
30            }
31        }
32    }
33
34    public int observerCount() {
35        return observers.size();
36    }
37
38 }
39

```

```
package com.pisethjava.observer.classic;
```

```
import java.util.List;
```

```
import java.util.concurrent.CopyOnWriteArrayList;
```

```
public class UserEventPublisher {
```

```
    // register observer
```

```
    private final List<UserObserver> observers = new CopyOnWriteArrayList<>();
```

```
    // CopyOnWriteArrayList<>() for thread safe
```

```
    public void subscribe(UserObserver observer) {
```

```
        if (observer == null) {
```

```
            throw new IllegalArgumentException("Observer is required");
```

```
        }
```

```
        observers.add(observer);
```

```
    }
```

```
    public void unsubscribe(UserObserver observer) {
```

```
        observers.remove(observer);
```

```
    }
```

```
    public void publish(UserRegisteredEvent event) {
```

```
        for (UserObserver observer : observers) {
```

```
            try {
```

```
                observer.onUserRegistered(event);
```

```
            } catch (RuntimeException exception) {
```

```
                System.out.printf("Observer %s failed: %s%n",
```

```
                    observer.getClass().getSimpleName(), exception.getMessage());
```

```
            }
```

```
        }
```

```
    }
```

```
    public int observerCount() {
```

```
        return observers.size();
```

```
    }
```

```
}
```